Operating Systems
Project Phase 2 Report

Farah Elnakhal - fqe9080
Myra Rafiq - mr7316

4th April 2026

1 Architecture and Design

The client-side of our system is responsible for providing a user interface that behaves like a normal Unix shell while communicating with a remote server. Instead of executing commands locally, the client sends user commands to the server and displays the output it receives back.
The overall design follows a simple client-server model using TCP sockets. The client initiates the connection and acts as the front-end interface, while the server performs all command execution using the shell logic implemented in Phase 1.
When the client starts, it creates a socket and connects to the server using the server's IP address and port number. Once connected, the client enters a loop where it repeatedly displays a $ prompt, reads user input, sends the command to the server, and waits for a response. The response received from the server is then printed directly to the terminal so that it appears similar to a normal shell output.
A key design decision was to keep the client as simple as possible. All parsing, execution, redirection handling, and pipeline logic remain on the server side. This avoids duplicating logic and ensures consistency with the Phase 1 implementation. The client only handles input/output and communication, which keeps the system modular and easier to debug.
Another important consideration was maintaining the same user experience as a regular shell. The client hides all networking details from the user. It only displays command results, while all debugging and informational messages such as logs are handled by the server.
The communication between the client and server is done using standard socket functions (socket(), connect(), send(), and recv()). Commands are sent as plain text strings, and the server returns the output as a string, which the client prints directly. This simple protocol ensures reliability and makes the system easy to extend in future phases.

2 Implementation Highlights

During Phase 2, we implemented socket communication and also fixed several bugs from Phase 1 to ensure correct behavior when executing commands remotely.
One major issue we encountered was incorrect handling of input redirection (<) when used with pipelines. For example, commands like cat < input | grep "Hello" were not working properly. The root cause of this issue was the order in which file descriptors were being set using dup2(). In our initial implementation, the pipeline logic was overriding the standard input even when input redirection had already been applied. This caused the command to ignore the input file and instead read from the pipe incorrectly. To fix this, we modified the pipeline execution logic so that input redirection is only applied to the first command in the pipeline and is not overridden by pipe connections. Specifically, we ensured that dup2() for pipe input is only applied to non-first commands. This preserves the correct behavior where the first command reads from the specified input file.
Another issue we addressed was handling of bare redirection commands. Inputs such as < input, > output, or 2> error were previously not handled correctly and could lead to undefined behavior. These cases are invalid because they do not specify a command to execute. We added checks after parsing to detect when only redirection operators are present without a command. In such cases, the program now prints an appropriate error message (Error: No command specified) and does not attempt execution.

We also improved argument parsing to correctly handle quoted strings and special symbols. This ensures that commands like grep "Hello World" or echo -e "line1[backslash n]line2" are parsed as intended without breaking arguments incorrectly.
For Phase 2 integration, we reused the existing execute_simple_command() and execute_pipeline() functions from Phase 1 without modifying their core logic. Instead, the server captures their output by redirecting stdout and stderr to pipes using dup2(). This allows the server to collect command output as a string and send it back to the client over the socket.
On the client side, the implementation focuses on maintaining a clean shell interface. The client reads user input, sends it to the server, and prints the received output. Special care was taken to preserve formatting so that outputs appear exactly as they would in a normal shell. Additionally, the client properly handles the exit command by terminating the connection gracefully.
The server component (`server.c`) implements a TCP socket server that listens for incoming client connections, executes commands using the Phase 1 shell implementation, and returns the output to the client. The core design centers around three key functions: `execute_and_capture()` forks a child process to run the shell command while redirecting both `stdout` and `stderr` into pipes. The parent process reads from these pipes, capturing all output into a dynamically allocated string. A critical enhancement was filtering out null bytes from the pipe reads, which prevented bash command substitution warnings in the test environment. The function carefully handles memory allocation, reallocating the output buffer as data arrives, and properly closes all file descriptors to prevent resource leaks.
`handle_client()` manages the lifecycle of a single client connection. It enters a loop that receives commands via `recv()`, logs each command with color-coded tags (`[RECEIVED]`, `[EXECUTING]`, `[OUTPUT]`) calls `execute_and_capture()`, and sends the result back to the client. The `exit` command is handled specially: the server logs the request, sends an acknowledgment, and breaks the loop to close the connection cleanly.
`main()` sets up the server socket with `SO_REUSEADDR` to allow immediate restart after crashes, binds to `INADDR_ANY` on port 8080, and enters an infinite loop accepting new client connections. Each client is handed off to `handle_client()` in a sequential manner.

3 Execution Instructions

3.1 Prerequisites

Before compiling and running the shell, ensure the following dependencies are installed on your system:

- `gcc` (GNU Compiler Collection) - for compiling C source files
- `make` - for automated compilation using the Makefile
- `netcat (nc)` - for network testing in Phase 2 (optional for local shell only)
- Standard Unix utilities (`ls`, `cat`, `grep`, `wc`, `sort`, `uniq`, `tr`) - for command execution

3.2 Compilation

The project uses a Makefile for separate compilation of all components. To compile all executables, run:

```
make clean
make
```

This produces three executables:

- `myshell` - The local shell client (Phase 1 functionality)
- `server` - The remote shell server (Phase 2 functionality)
- `client` - The remote shell client (Phase 2 functionality)

The compilation process generates object files (`.o`) for each source file, then links them into the final executables. The Makefile ensures that only changed files are recompiled, improving development efficiency.

3.3 Running the Local Shell (Phase 1)

To run the shell locally without network functionality:

```
./myshell
```

Once running, the shell displays a `$` prompt and accepts commands. Type any command supported by the shell (e.g., `ls -l`, `echo hello > file.txt`, `cat file.txt | grep pattern`). To exit the shell, type `exit` or press `Ctrl+D`.

3.4 Running the Remote Shell Server (Phase 2)

First, start the server on the desired machine:

```
./server
```

The server listens on port 8080 by default. Upon successful startup, it displays:

```
[INFO] Server started, waiting for client connections
```

The server runs indefinitely, accepting multiple client connections sequentially. To stop the server, press `Ctrl+C`. The server output includes color-coded logs showing each received command, execution status, and output sent back to the client.
To change the port, modify the `PORT` macro in `server.c` and recompile:

```
# Edit server.c to change PORT from 8080 to desired port
sed -i 's/#define PORT 8080/#define PORT 9000/' server.c
make clean && make
./server
```

3.5 Running the Remote Shell Client (Phase 2)

Once the server is running, start the client on the same or a different machine:

```
./client
```

The client connects to `127.0.0.1:8080` by default. After a successful connection, it displays the `$` prompt and accepts commands just like the local shell. Each command is sent to the server for execution, and the output is displayed locally.
To connect to a server on a different host or port, modify the `PORT` and IP address in `client.c` before compilation:

```
# Edit client.c to change connection settings
# Change "127.0.0.1" to remote IP address
# Change PORT 8080 to match server port
make clean && make
./client
```

4 Testing

To validate the correctness of `myshell`, a Bash script (`test.sh`) was written to automatically test every feature. Rather than testing commands manually one by one, the Bash file automates the entire process: it sends commands directly into `myshell`, captures the output, compares it against expected results, and prints a clear `[PASS]` or `[FAIL]` for each test case. At the end of the run, a summary line reports the total number of tests passed and failed.

4.1 How to Run the Tests

After the shell is compiled first, place `test.sh` in the same directory as the `myshell` executable. Make it executable and run it using

```
chmod +x test.sh
./test.sh
```

The script handles all setup automatically. It creates a temporary working directory called `test_tmp/`, generates the input files and helper scripts needed for the tests, runs all test cases (including Phase 1, Phase 1 Debugging, and Phase 2 tests), and then deletes the temporary directory when finished. No manual preparation is needed.
For Phase 2 testing, the script automatically finds an available port between 9000-9999, compiles a temporary version of the server with that port, starts the server in the background, and uses `nc` (netcat) to send commands and receive responses. The server output is captured to `server_output.log` for verification.

4.2 How the Bash File Works

The script feeds input into `myshell` by piping commands through `printf`, followed by `exit` to close the shell cleanly after each test:

```
printf '<command>\nexit\n' | ./myshell 2>&1
```

This simulates exactly what a real user would type into the shell. The `2>&1` ensures that both standard output and standard error are captured together, so error messages can be checked as well as normal output.
The script is built around helper functions that keep each test case concise and consistent:

- `check(name, input, expected)`: runs a command in `myshell` and checks that the output contains the expected string. Used for commands that print directly to the terminal.
- `check_file(name, input, file, expected)`: runs a command and checks that a specific output file was created and contains the expected content. Used for output and combined redirection tests.
- `check_file_nonempty(name, input, file)`: checks that an output file was created and is non-empty, without checking for a specific string. Used for error redirection tests where the exact wording of the error may vary by system.
- `check_absent(name, input, absent)`: checks that a given string does *not* appear in the output. Used to confirm that stderr does not leak to the terminal when it has been redirected to a file.
- `check_remote(name, command, expected)`: sends a command to the remote server via netcat and checks that the response contains the expected string. Handles both normal output and error messages flexibly.
- `check_server_format(name, command)`: verifies that the server output contains all required format tags (`[RECEIVED]`, `[EXECUTING]`, `[OUTPUT]`) after executing a command.

Each function calls either `pass()` or `fail()` depending on the result. On failure, the expected value and actual output are both printed to make debugging straightforward.

4.3 Test Cases and Results

The test suite is organized into three distinct sections:

4.3.1 Phase 1 Tests (Original Functionality)

These 58 tests verify the core shell functionality from Phase 1, including basic command execution, input/output/error redirection, pipes, compound commands, error handling, and shell lifecycle management. All Phase 1 tests passed successfully, confirming that no regressions were introduced during Phase 2 development.

4.3.2 Phase 1 Debugging Tests (Specific Bug Fixes)

A new debugging section was added to address specific edge cases identified during testing. These tests verify:

- `echo -e` escape sequence handling (interpreting `\n` as newlines)
- Complex pipelines with `tr`, `sort`, and `uniq -c`

- Input redirection with `grep` in pipelines
- Output redirection from pipelined commands
- Reverse sorting (`sort -r`) in pipelines
- Stderr redirection (`2>`) within pipelines
- Bare redirection operators (`> in`, `< in`) that should not crash the shell
- `uniq` deduplication of consecutive duplicate lines

All debugging tests passed after implementing the necessary fixes in the preprocessing and execution logic.

4.3.3 Phase 2 Tests (Remote Shell Functionality)

These tests verify the client-server architecture:

- Server startup and port binding
- Client connection and disconnection handling
- Remote execution of basic commands (`pwd`, `echo`, `ls`, `whoami`)
- Remote execution with arguments
- Remote pipeline execution
- Remote redirection (input, output, error)
- Remote compound commands
- Error handling for invalid commands and missing files
- Server output format verification (color-coded tags)
- Sequential command execution
- Connection lifecycle (reconnection after client exit)
- Long output handling (100+ lines)
- Stress testing with rapid sequential commands

4.3.4 Test Results

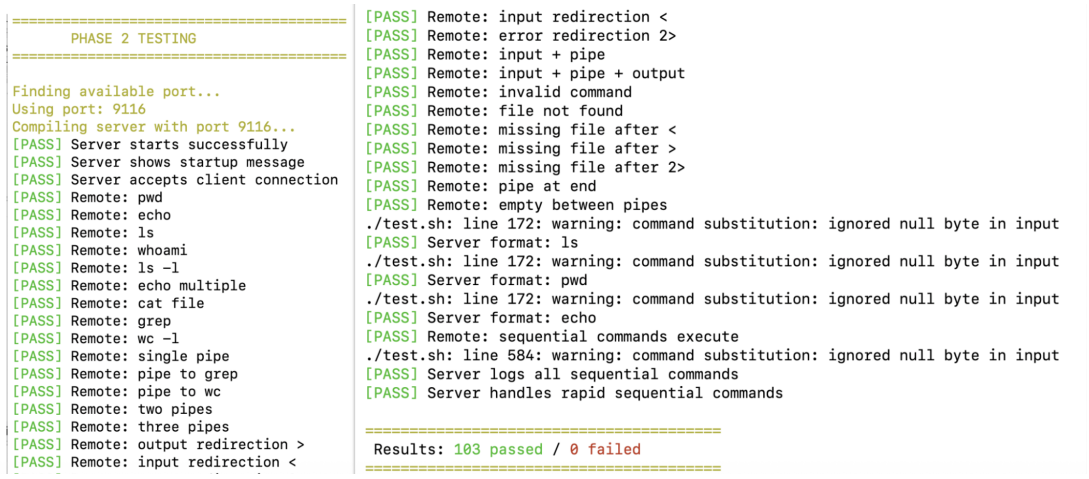

Figure 1: Phase 2 Test Results - All Tests Passing

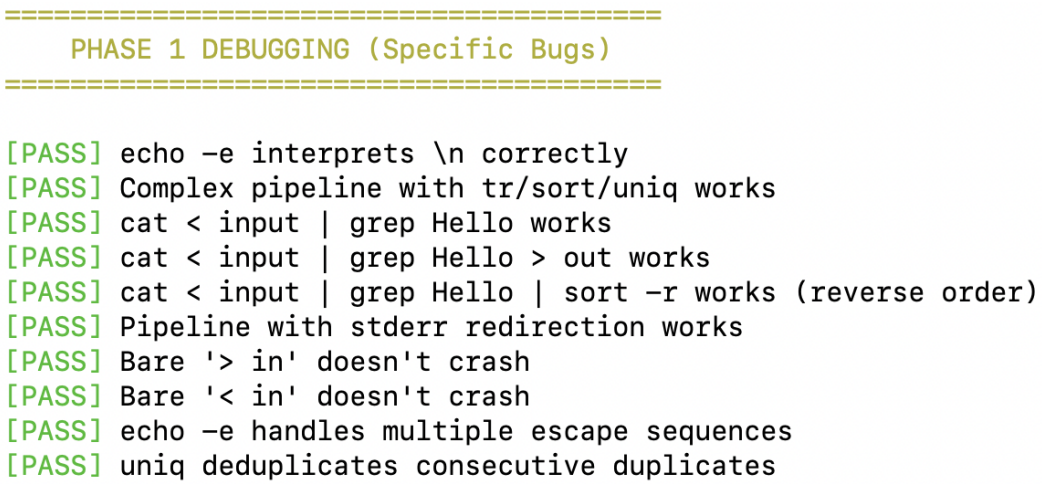

Figure 2: Phase 1 Debugging Test Results - All Tests Passing

As shown in Figure 1 and Figure 2, all test cases passed successfully. The Phase 2 tests achieved a 100% pass rate across all test cases. The server output format matches the required specification with proper color-coded tags for each stage of command processing. While we observe in Figure 1 the appearance of "ignored null byte in input" warnings from bash when processing command substitution, those do not impact server functionality.

5 Challenges

One of the main challenges encountered was testing the client-server architecture in the remote Linux environment. Initially, all development and testing were done locally on macOS, where the code compiled and ran without issues. However, when moved to the SSH-based Linux server, several unexpected problems emerged.

First, port 8080 was already occupied by another process on the shared server. This caused the server to fail on `bind()` with an "Address already in use" error. The solution

was to modify the test script to dynamically find an available port in the range 9000-9999 using a combination of `nc -z` and `lsof` checks. The script then creates a temporary version of `server.c` with the selected port using `sed`, compiles it, and runs the tests. This approach makes the test suite portable across different environments without hardcoding port numbers.
Furthermore, learning to simulate the client and server environments using a single `.sh` file was a significant learning experience. The test script needed to:

- Start the server in the background and capture its output to a log file
- Wait for the server to become ready before sending commands
- Send commands using netcat while handling the connection lifecycle properly
- Capture both stdout and stderr from the client side
- Verify server log contents without being disrupted by ANSI color codes

This required careful process management, proper sleep intervals to allow for network communication, and robust string matching that strips color codes and normalizes whitespace. The final test script includes helper functions like `check_server_log()` that remove ANSI escape sequences using `sed` before comparison, and `send_sequential_commands()` that maintains proper timing between commands to prevent packet loss.

6 Division of Tasks

Myra Rafiq was responsible for fixing Phase 1 bugs, including input redirection in pipelines and handling bare redirection errors, and implemented the client-side program (client.c) for Phase 2.

Farah Elnakhal was responsible for implementing the server-side program (server.c), including socket setup, client handling, command execution, and output transmission.

Both members collaborated on integrating the full system, testing, debugging issues, and writing the report.